

Symbiosis Institute of Technology, Pune

Faculty of Engineering

CSE- Academic Year 2026-27

Java Programming – Lab Batch 2025-29

Lab Assignment No:- 26	
Name of Student	Arpit Chakraborty
PRN No.	25070122047
Batch	2025-29
Class	CSE A2
Academic Year & Semester	26-27, III
Date of Submission	27/10/26
Title of Assignment:	Write a Java program to create and execute multiple threads using the Runnable interface.

Practice questions
 	
Output Screenshots Solve questions on given link and attach screen shot of leetcode with your login and o/p
	1. Create a program using Runnable to run two tasks simultaneously.
	class Task1 implements Runnable {
	public void run() {
	for (int i = 1; i <= 5; i++) {
	System.out.println("Task 1: " + i);
	}
	}
	}
	
	class Task2 implements Runnable {
	public void run() {
	for (int i = 1; i <= 5; i++) {

```
            System.out.println("Task 2: " + i);
        }
    }
}

public class MultipleThreads {
    public static void main(String[] args) {
        Task1 t1 = new Task1();
        Task2 t2 = new Task2();

        Thread thread1 = new Thread(t1);
        Thread thread2 = new Thread(t2);

        thread1.start();
        thread2.start();
    }
}
```

2. Create a Java program using the Runnable interface where two tasks run simultaneously: one thread performs deposit operations, and another performs withdrawal operations, both displaying results concurrently.

```
class BankAccount {
    int balance = 1000;

    synchronized void deposit(int amount) {
        balance += amount;
        System.out.println("Deposited: " + amount);
        System.out.println("Balance: " + balance);
    }

    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
```

```
            System.out.println("Withdrawn: " + amount);
            System.out.println("Balance: " + balance);
        } else {
            System.out.println("Insufficient balance!");
        }
    }
}

class DepositTask implements Runnable {
    BankAccount account;

    DepositTask(BankAccount account) {
        this.account = account;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            account.deposit(100);
        }
    }
}

class WithdrawTask implements Runnable {
    BankAccount account;

    WithdrawTask(BankAccount account) {
        this.account = account;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            account.withdraw(100);
        }
    }
```

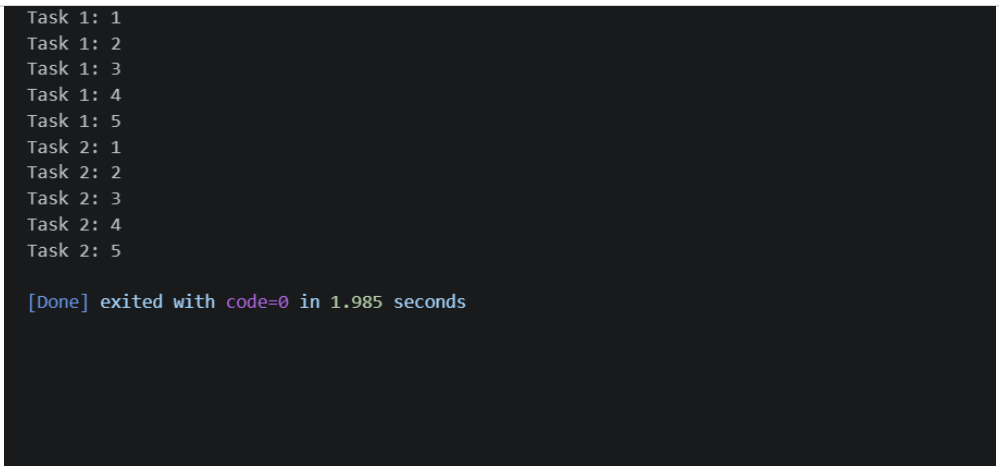
```
}

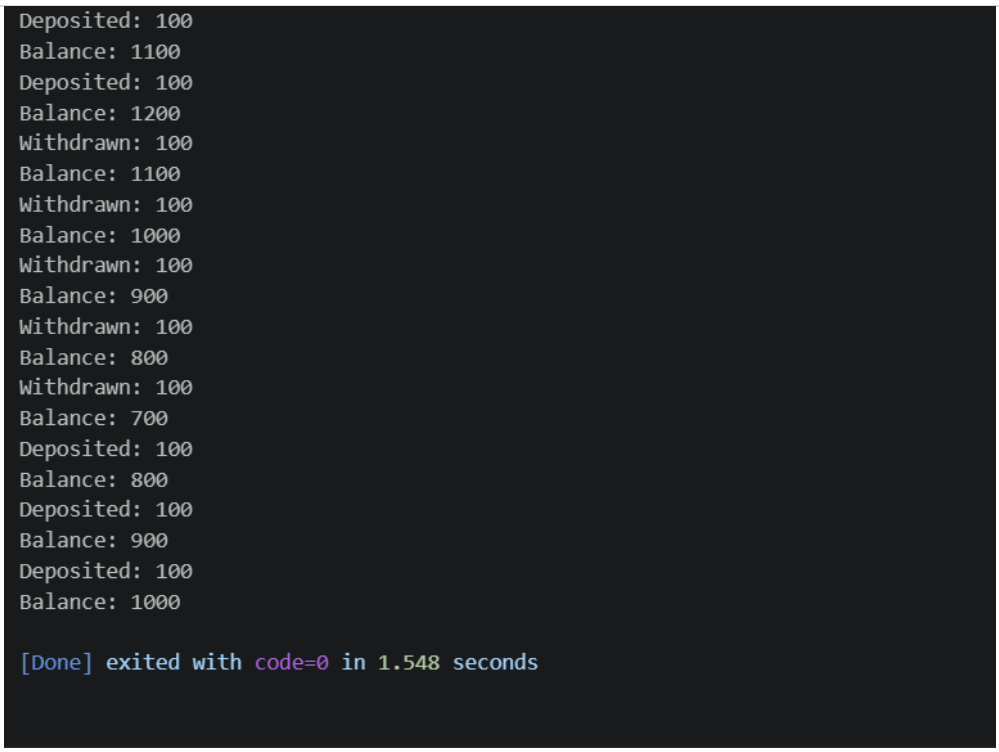
public class BankDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        Thread t1 = new Thread(new DepositTask(account));
        Thread t2 = new Thread(new WithdrawTask(account));

        t1.start();
        t2.start();
    }
}
```

Output Screenshots (if applicable)



	
Conclusion	Thus, we successfully created and executed multiple threads using the Runnable interface in Java. We learned to perform two tasks concurrently and implement deposit and withdrawal operations using threads. We also understood the use of Thread, start(), and synchronized for safe multithreaded execution.